Diagrams

15 March 2026 16:27

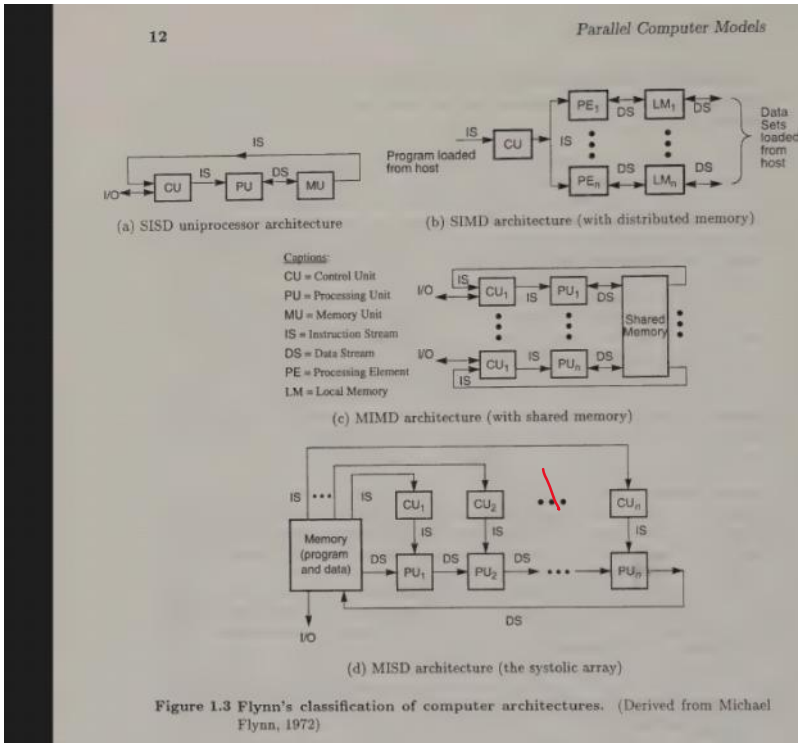


Figure 1.3 Flynn's classification of computer architectures. (Derived from Michael Flynn, 1972)

Implementation models and case studies of SIMD machines are given in Chapter 8.

SIMD Machine Model An operational model of an SIMD computer is specified by a 5-tuple:

$$M = \langle N, C, I, M, R \rangle \tag{1.5}$$

where

(1) N is the number of *processing elements* (PEs) in the machine. For example, the Illiac IV has 64 PEs and the Connection Machine CM-2 uses 65,536 PEs.

(2) C is the set of instructions directly executed by the *control unit* (CU), including scalar and program flow control instructions.

(3) I is the set of instructions broadcast by the CU to all PEs for parallel execution. These include arithmetic, logic, data routing, masking, and other local operations executed by each active PE over data within that PE.

(4) M is the set of masking schemes, where each mask partitions the set of PEs into enabled and disabled subsets.

(5) R is the set of data-routing functions, specifying various patterns to be set up in the interconnection network for inter-PE communications.

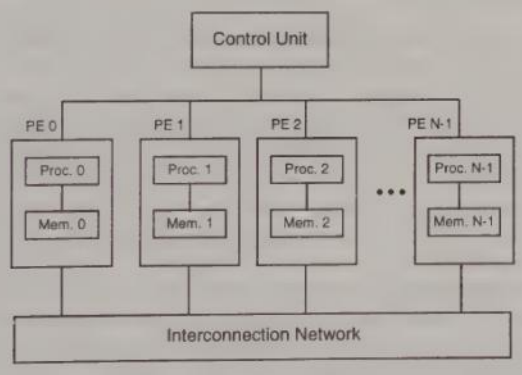


Figure 1.12 Operational model of SIMD computers.

One can describe a particular SIMD machine architecture by specifying the 5-tuple. An example SIMD machine is partially specified below:

② Explicit parallelism

Implicit parallelism

Programmer

↓

Source code written in sequential langs. like C, Fortran etc.

↓

parallelising compiler

↓

parallel object code

↓

Execute by Runtime system

Good Write

DATE: __/__/____
PAGE: _____

→ Explicit parallelism

program

↓

source code written in concurrent language like C, Fortran, Pascal

→ using multiple threads (like Java)

↓

concurrency preserving compiler

↓

Concurrent object code

↓

Execute by Runtime System

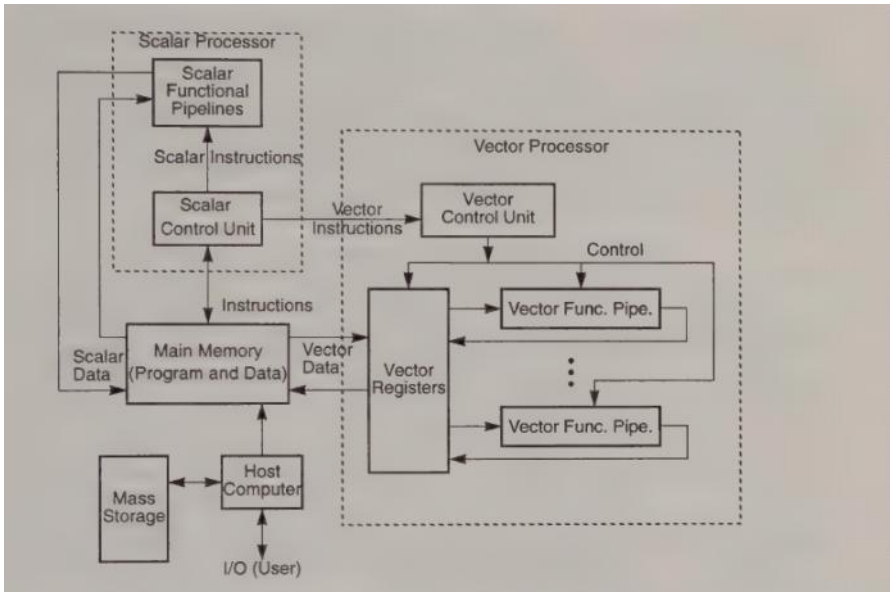


Figure 1.11 The architecture of a vector supercomputer.

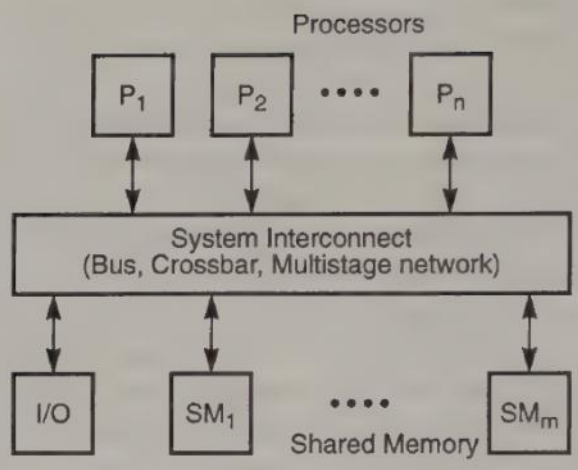
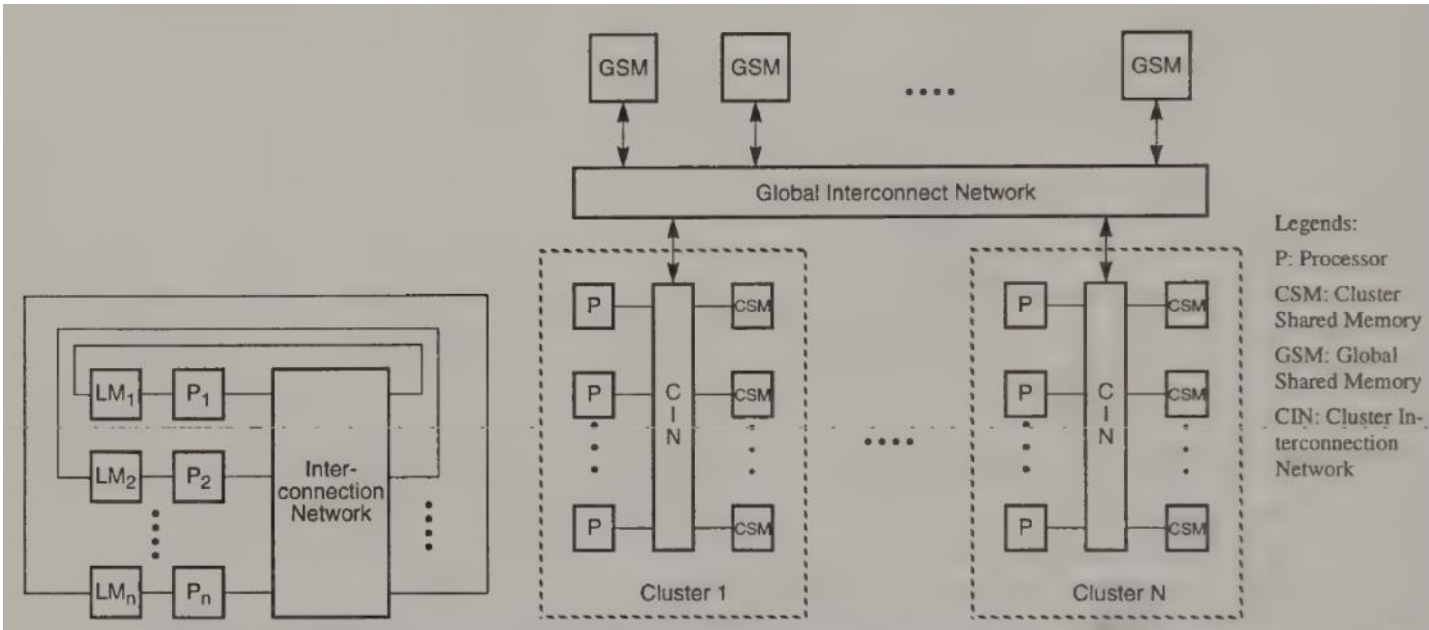


Figure 1.6 The UMA multiprocessor model (e.g., the Sequent Symmetry S-81).



(a) Shared local memories (e.g., the BBN Butterfly)

(b) A hierarchical cluster model (e.g., the Cedar system at the University of Illinois)

Figure 1.7 Two NUMA models for multiprocessor systems.

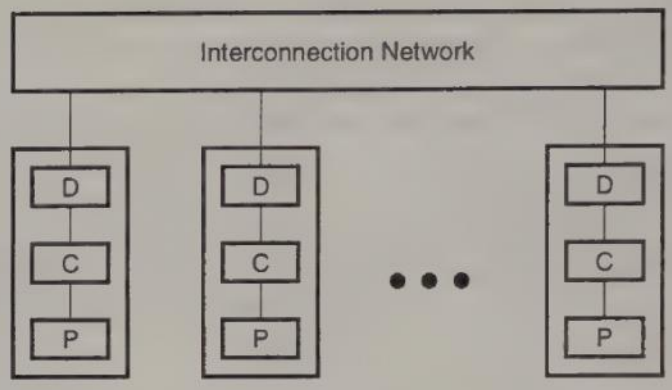


Figure 1.8 The COMA model of a multiprocessor. (P: Processor, C: Cache, D: Directory; e.g., the KSR-1)

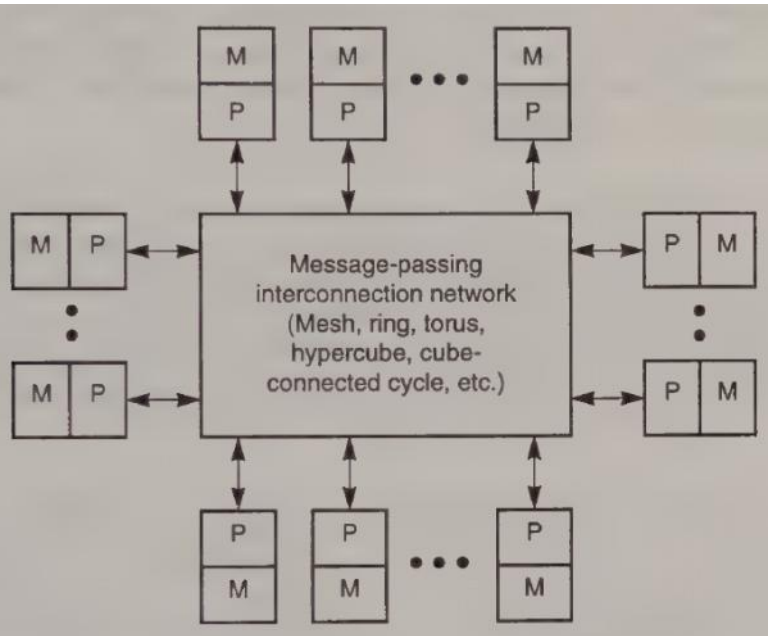


Figure 1.9 Generic model of a message-passing multicomputer.

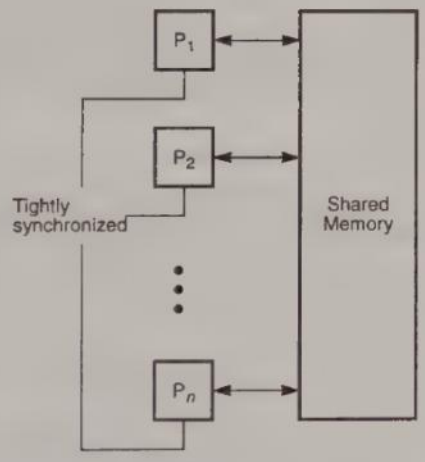


Figure 1.14 PRAM model of a multiprocessor system with shared memory, on which all n processors operate in lockstep in memory access and program execution operations. Each processor can access any memory location in unit time.

Example 1.5 Multiplication of two $n \times n$ matrices in $O(\log n)$ time on a PRAM with $n^3/\log n$ processors (Viktor Prasanna, 1992)

Let A and B be the input matrices. Assume n^3 PEs are available initially. We later reduce the number of PEs to $n^3/\log n$. To visualize the algorithm, assume the memory is organized as a three-dimensional array with inputs A and B stored in two planes. Also, for sake of explanation, assume a three-dimensional indexing of the PEs. PE(i, j, k), $0 \le k \le n-1$ are used for computing the (i, j)th entry of the output matrix, $0 \le i, j \le n-1$, and n is a power of 2.

In step 1, n product terms corresponding to each output are computed using n PEs in $O(1)$ time. In step 2, these are added to produce an output in $O(\log n)$ time.

The total number of PEs used is n^3. The result is available in $C(i, j, 0), 0 \le i, j \le n-1$. Listed below are programs for each PE(i, j, k) to execute. All n^3 PEs operate in parallel for n^3 multiplications. But at most $n^3/2$ PEs are busy for $(n^3 - n^2)$ additions. Also, the PRAM is assumed to be CREW.

```
Step 1:
  1. Read A(i,k)
  2. Read B(k,j)
  3. Compute A(i,k) × B(k,j)
  4. Store in C(i,j,k)

Step 2:
  1. ℓ ← n
  2. Repeat
        ℓ ← ℓ/2
        if (k < ℓ) then
           begin
              Read C(i,j,k)
              Read C(i,j,k+ℓ)
              Compute C(i,j,k) + C(i,j,k+ℓ)
              Store in C(i,j,k)
           end
     until (ℓ = 1)
```

To reduce the number of PEs to $n^3/\log n$, use a PE array of size $n \times n \times n/\log n$. Each PE is responsible for computing $\log n$ product terms and summing them up. Step 1 can be easily modified to produce $n/\log n$ partial sums, each consisting of $\log n$ multiplications and $(\log n - 1)$ additions. Now we have an array C(i, j, k), $0 \le i, j \le n-1, 0 \le k \le n/\log n - 1$, which can be summed up in $\log(n/\log n)$ time. Combining the time spent in step 1 and step 2, we have a total execution time $2\log n - 1 + \log(n/\log n) \approx O(\log n)$ for large n.

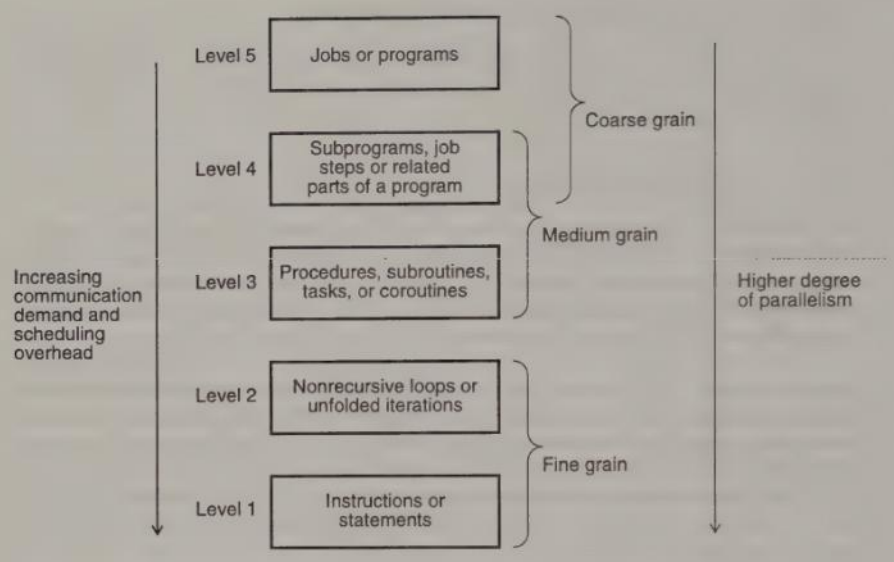
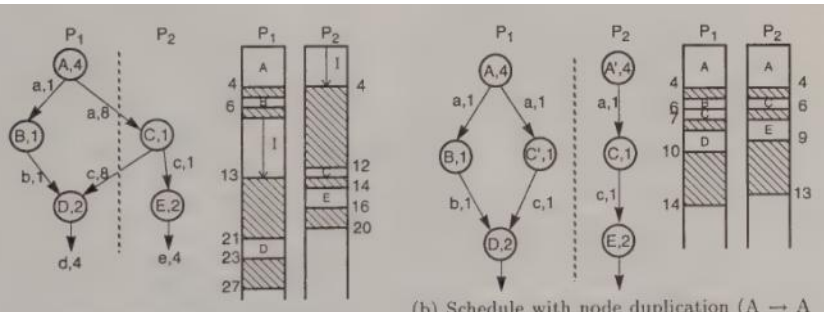


Figure 2.5 Levels of parallelism in program execution on modern computers. (Reprinted from Hwang, *Proc. IEEE*, October 1987)

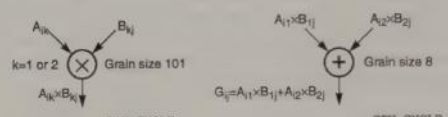


(a) Schedule without node duplication

(b) Schedule with node duplication (A → A and A′; C → C and C′)

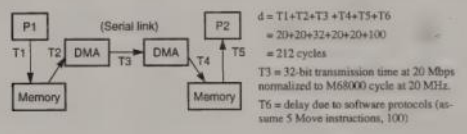
Figure 2.8 Node-duplication scheduling to eliminate communication delays between processors. (I: idle time; shaded areas: communication delays)

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$




```
                    CPU CYCLE                          CPU CYCLE
Move W   Axx, D1    15          Move L   PAR1,D1    20
Move W   Bxx, D2    15          Move L   PAR2,D2    20
MPTY D1, D2         71          ADD  L   D1,  D2    8
MOVE L   D2,  PAR   20          MOVE L   D2,  PSUM  20
```

(a) Grain size calculation in M68000 assembly code at 20-MHz cycle

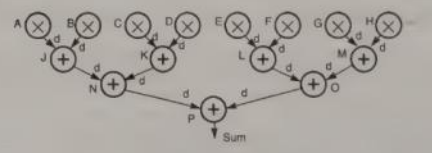


d = T1+T2+T3 +T4+T5+T6
= 20+20+32+20+20+100
= 212 cycles

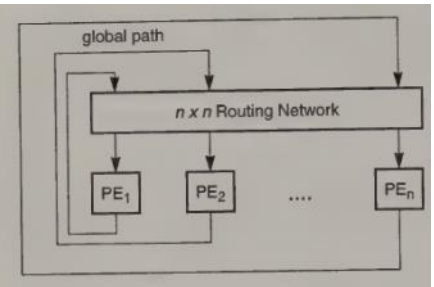
T3 = 32-bit transmission time at 20 Mbps normalized to M68000 cycle at 20 MHz.

T6 = delay due to software protocols (assume 5 Move instructions, 100)

(b) Calculation of communication delay d



(c) Fine-grain program graph



(a) The global architecture

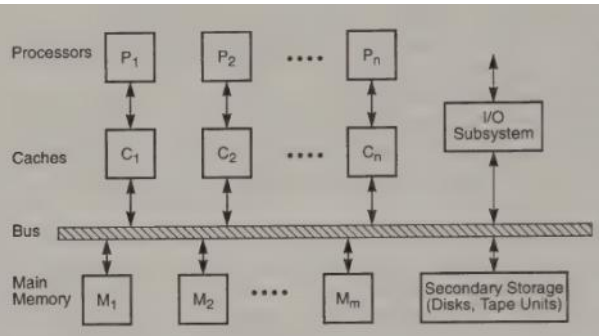


Figure 2.22 A bus-connected multiprocessor system, such as the Sequent Symmetry S1.

Multistage Networks MINs have been used in both MIMD and SIMD computers. A generalized multistage network is illustrated in Fig. 2.23. A number of $a \times b$ switches are used in each stage. Fixed interstage connections are used between the switches in adjacent stages. The switches can be dynamically set to establish the desired connections between the inputs and outputs.

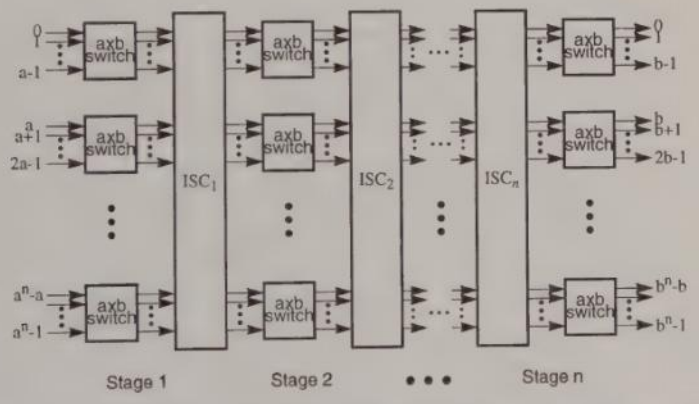
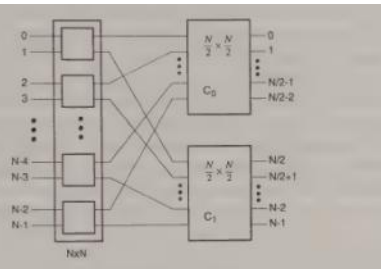
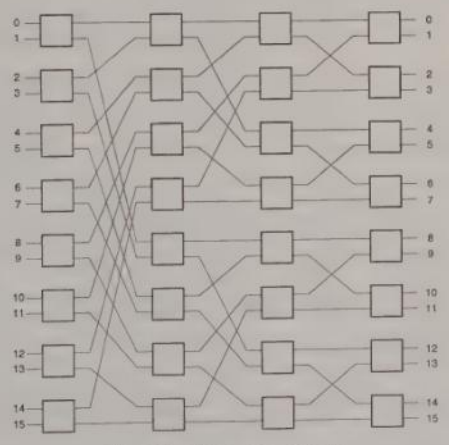


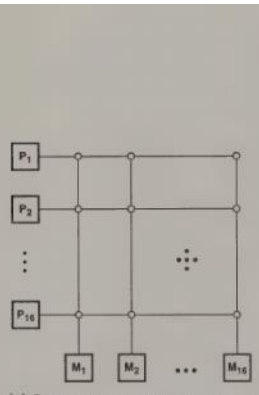
Figure 2.23 A generalized structure of a multistage interconnection network (MIN) built with a × b switch modules and interstage connection patterns $ISC_1, ISC_2, ..., ISC_n$.



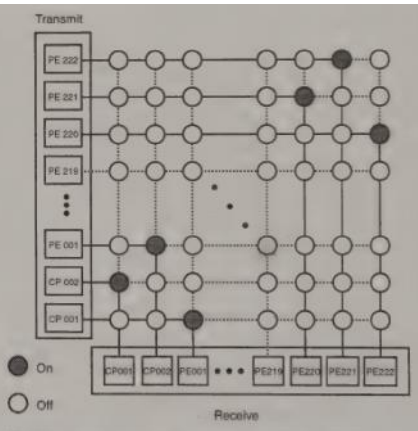
(a) Recursive construction



(b) A 16 × 16 Baseline network



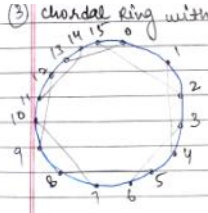
(a) Interprocessor-memory crossbar network built in the C.mmp multiprocessor at Carnegie-Mellon University (1972)



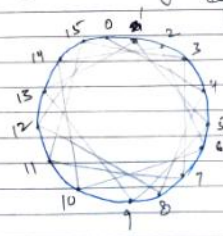
(b) The interprocessor crossbar network built in the Fujitsu VPP500 vector parallel processor (1992)

Figure 2.26 Two crossbar switch network configurations.

(3) chordal Ring with deg = 3



(4) chordal Ring with deg = 4

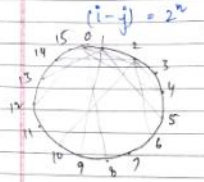


Good Write

(5)

For a N/w of 16 Nodes, Barrel shifter is obtained from the Ring by adding extra links from each node to those Node having a distance equal to an integer power of 2.

$(i-j) = 2^n$; $n = 0, 1, 2, 3 \ldots$



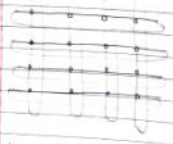
N/w size $N = 2^n$

Diameter $= N/2$

degree $= 2n-1$

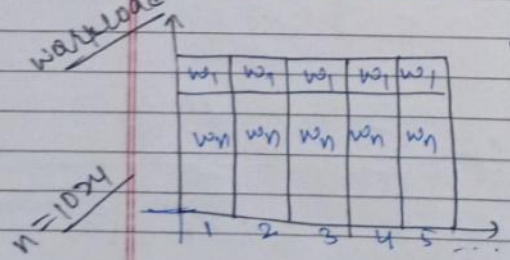
(6) Star (7) Tree (8) Fat Trees

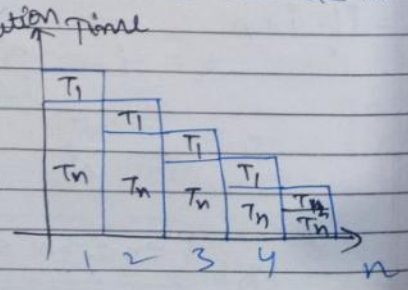
(9) Three Network (10) Illiac map



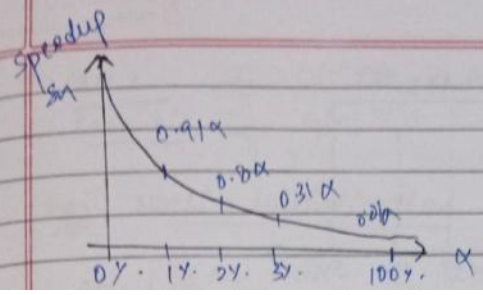
(11) Hypercube (12) Mesh

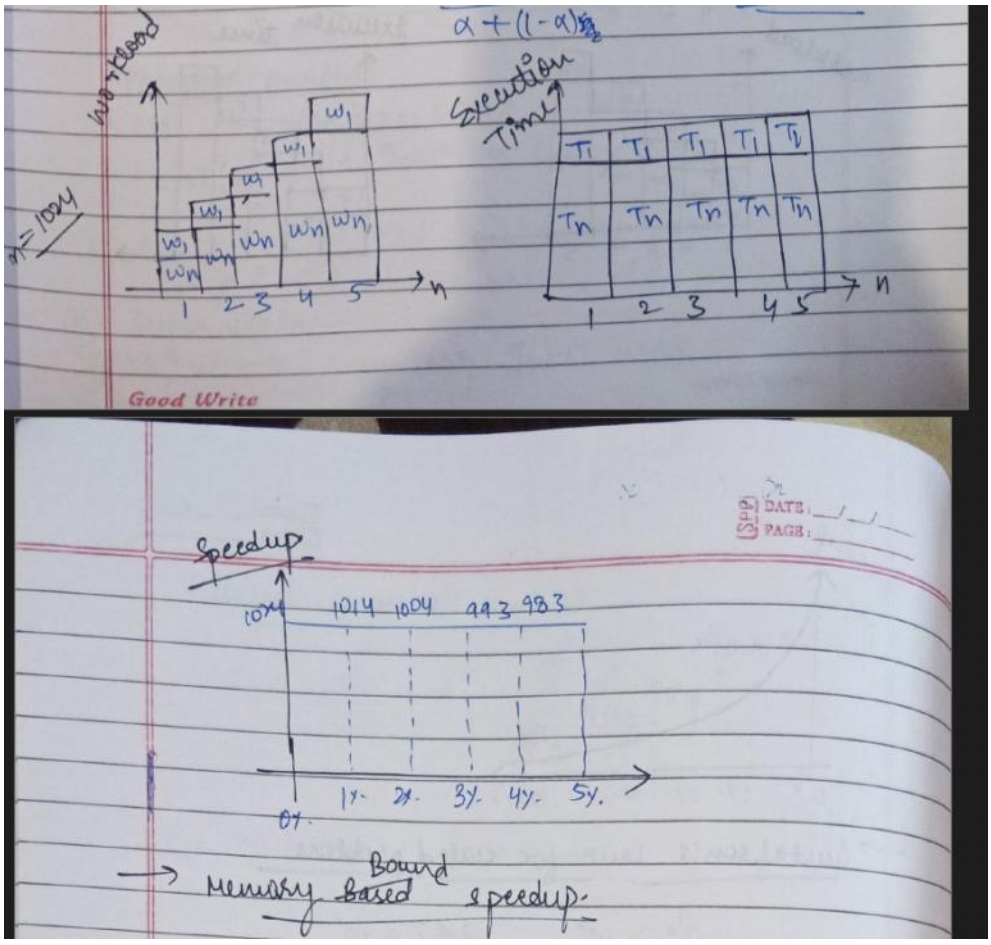
"α is called sequential Bottleneck in pgm





Good Write





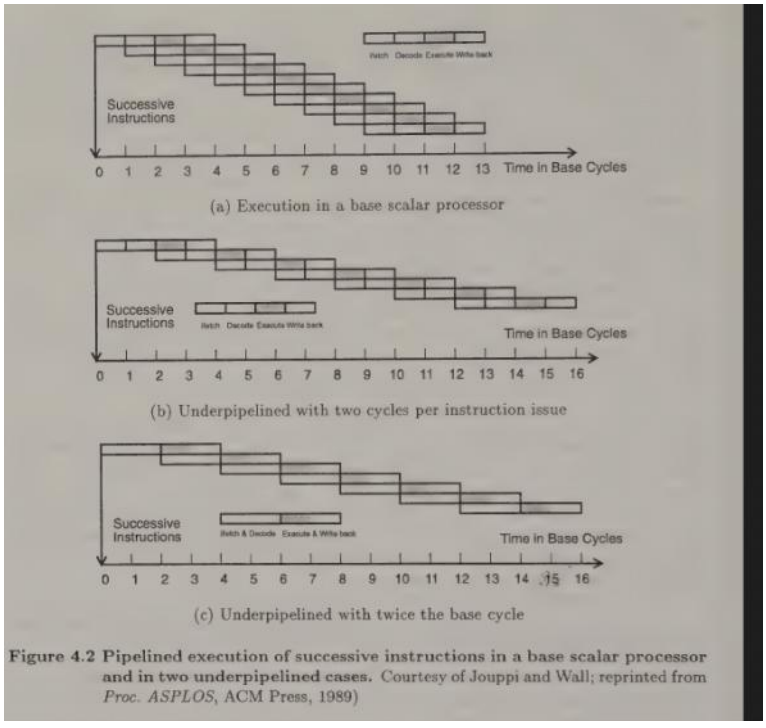
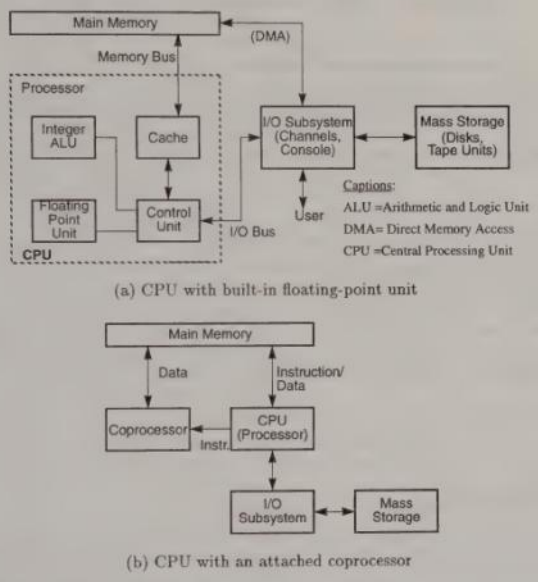


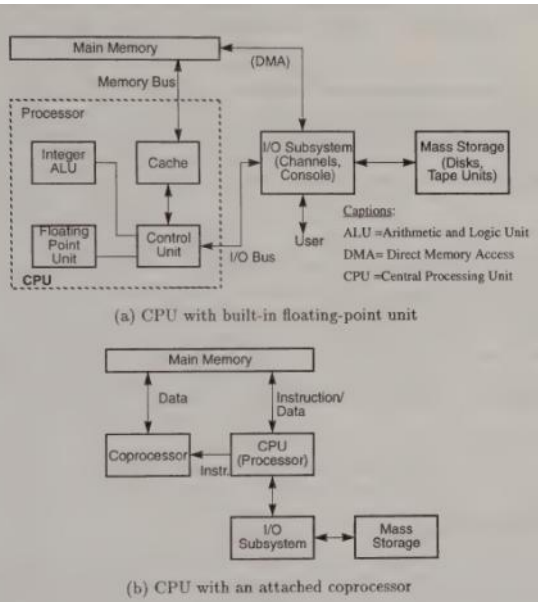
Figure 4.2 Pipelined execution of successive instructions in a base scalar processor and in two underpipelined cases. Courtesy of Jouppi and Wall; reprinted from *Proc. ASPLOS*, ACM Press, 1989)



(a) CPU with built-in floating-point unit

(b) CPU with an attached coprocessor

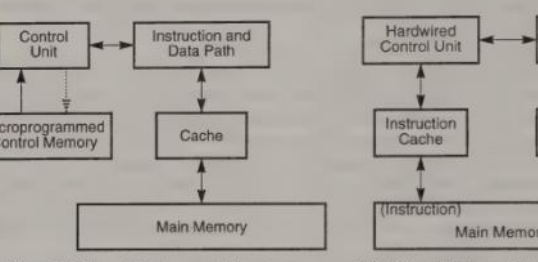
Figure 4.3 Architectural models of a basic scalar computer system.



(a) CPU with built-in floating-point unit

(b) CPU with an attached coprocessor

Figure 4.3 Architectural models of a basic scalar computer system.



(a) The CISC architecture with microprogrammed control and unified cache

(b) The RISC architecture with hardwired control and split instruction cache and data cache.

igure 4.4 Distinctions between typical RISC and typical CISC processor architectures. (Courtesy of Gordon Bell, 1989)

Pipelining in Superscalar Processors The fundamental structure of a superscalar pipeline is illustrated in Fig. 4.11. This diagram shows the use of three instruction pipelines in parallel for a triple-issue processor. Superscalar processors were originally developed as an alternative to vector processors.

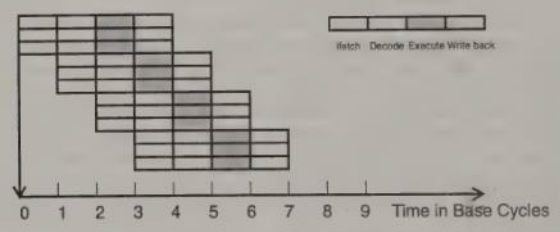
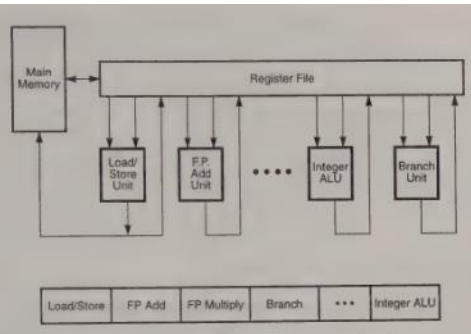
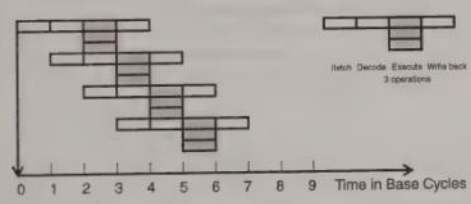


Figure 4.11 A superscalar processor of degree m = 3.



(a) A typical VLIW processor and instruction format



(b) VLIW execution with degree $m = 3$

Figure 4.14 The architecture of a very long instruction word (VLIW) processor and its pipeline operations. (Courtesy of Multiflow Computer, Inc., 1987)

The latter uses memory-based instructions which are longer in length, including memory addresses.

Vector Instructions Register-based vector instructions appear in most register-to-register vector processors like Cray supercomputers. Denote a *vector register* of length n as $\mathbf{V}_i$, a *scalar register* as s_i, and a *memory array* of length n as $M(1:n)$. Typical register-based vector operations are listed below, where a vector operator is denoted by a small circle "$\circ$":

$$
\begin{array}{lcll}
\mathbf{V}_1 \circ \mathbf{V}_2 & \longrightarrow & \mathbf{V}_3 & \text{(binary vector)} \\
s_1 \circ \mathbf{V}_1 & \longrightarrow & \mathbf{V}_2 & \text{(scaling)} \\
\mathbf{V}_1 \circ \mathbf{V}_2 & \longrightarrow & s_1 & \text{(binary reduction)} \\
M(1:n) & \longrightarrow & \mathbf{V}_1 & \text{(vector load)} \\
\mathbf{V}_1 & \longrightarrow & M(1:n) & \text{(vector store)} \\
\circ\, \mathbf{V}_1 & \longrightarrow & \mathbf{V}_2 & \text{(unary vector)} \\
\circ\, \mathbf{V}_1 & \longrightarrow & s_1 & \text{(unary reduction)}
\end{array}
\tag{4.1}
$$

It should be noted that the vector length should be equal in all operands used in a vector instruction. The reduction is an operation on one or two vector operands, and the result is a scalar — such as the *dot product* between two vectors and the *maximum* of all components in a vector.

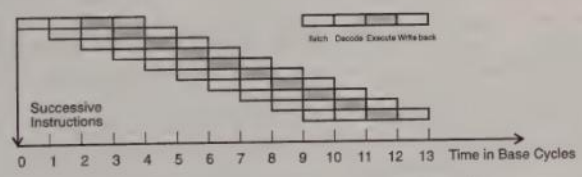
In all cases, these vector operations are performed by dedicated pipeline units, including *functional pipelines* and *memory-access pipelines*. Long vectors exceeding the register length n must be segmented to fit the vector registers n elements at a time.

Memory-based vector operations are found in memory-to-memory vector processors

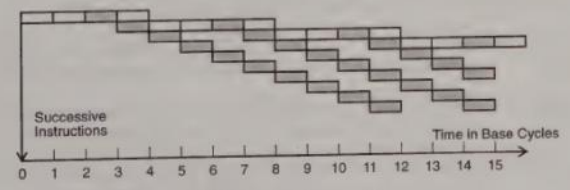
such as those in the Cyber 205. Listed below are a few examples:

$$
\begin{array}{rcll}
M_1(1:n) & \circ & M_2(1:n) & \longrightarrow M(1:n) \\
s_1 & \circ & M_1(1:n) & \longrightarrow M_2(1:n) \\
 & \circ & M_1(1:n) & \longrightarrow M_2(1:n) \\
M_1(1:n) & \circ & M_2(1:n) & \longrightarrow M(k)
\end{array}
\tag{4.2}
$$

compiler that vectorizes sequential code for vector pipelining.

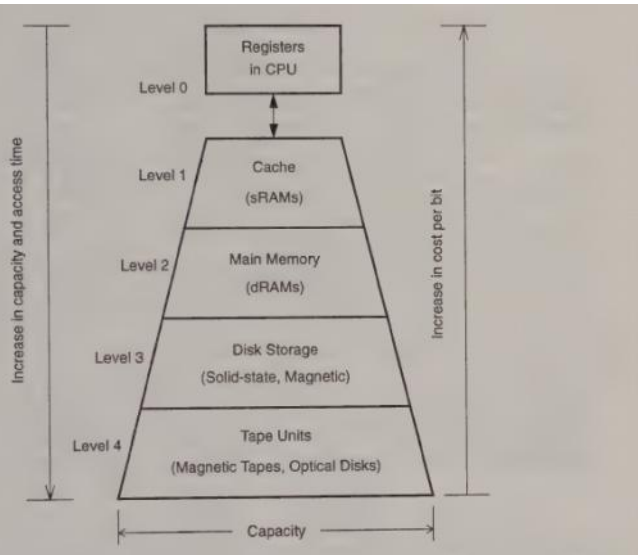


(a) Scalar pipeline execution (Fig. 4.2a redrawn)

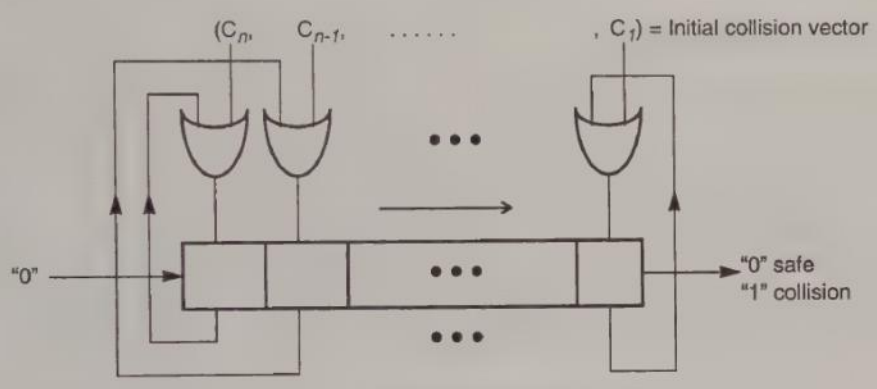


(b) Vector pipeline execution

Figure 4.15 Pipelined execution in a base scalar processor and in a vector processor, respectively. (Courtesy of Jouppi and Wall; reprinted from *Proc. ASPLOS*, ACM Press, 1989)



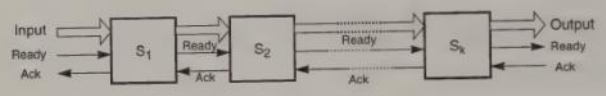
e 4.17 A four-level memory hierarchy with increasing capacity and decreasing speed and cost from low to high levels.



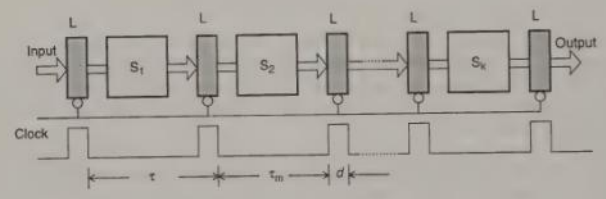
(a) State transition using an n-bit right shift register, where n is the maximum forbidden latency

Asynchronous Model As shown in Fig. 6.1a, data flow between adjacent stages in an asynchronous pipeline is controlled by a handshaking protocol. When stage S_i is ready to transmit, it sends a *ready* signal to stage S_{i+1}. After stage S_{i+1} receives the incoming data, it returns an *acknowledge* signal to S_i.

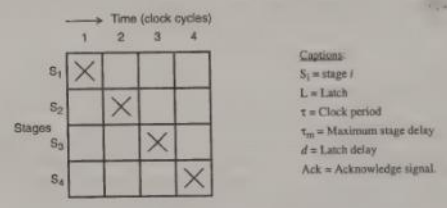
Asynchronous pipelines are useful in designing communication channels in message-passing multicomputers where pipelined wormhole routing is practiced (see Chapter 9). Asynchronous pipelines may have a variable throughput rate. Different amounts of delay may be experienced in different stages.



(a) An asynchronous pipeline model

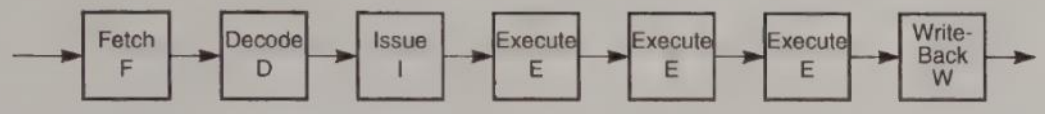


(b) A synchronous pipeline model



(c) Reservation table of a four-stage linear pipeline

Figure 6.1 Two models of linear pipeline units and the corresponding reservation table.



(a) A seven-stage instruction pipeline

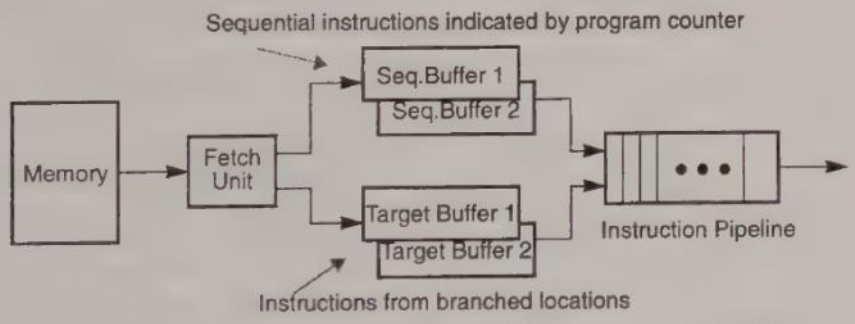
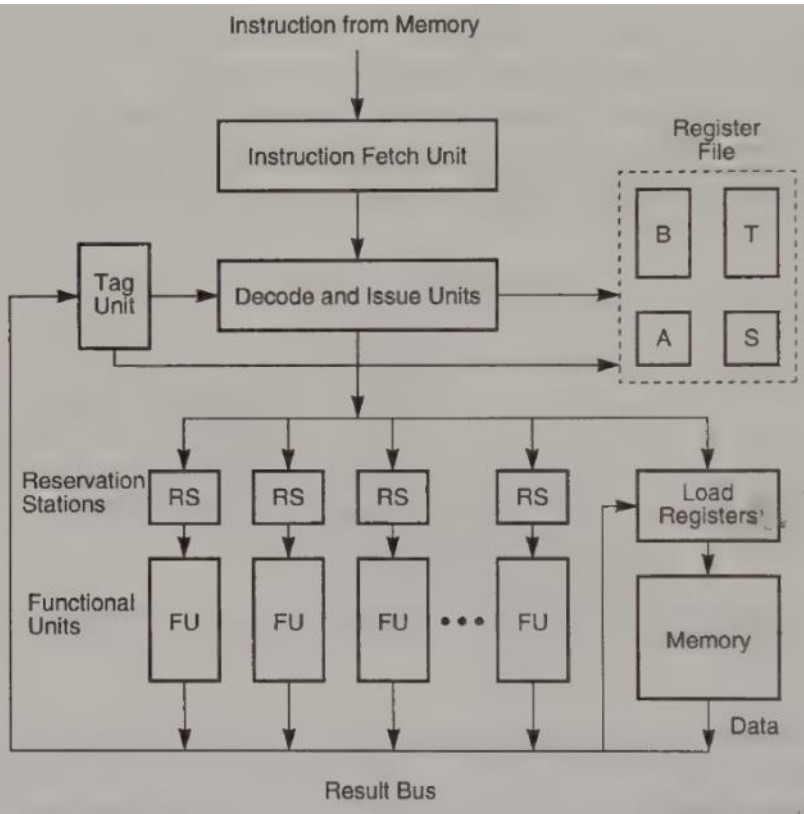
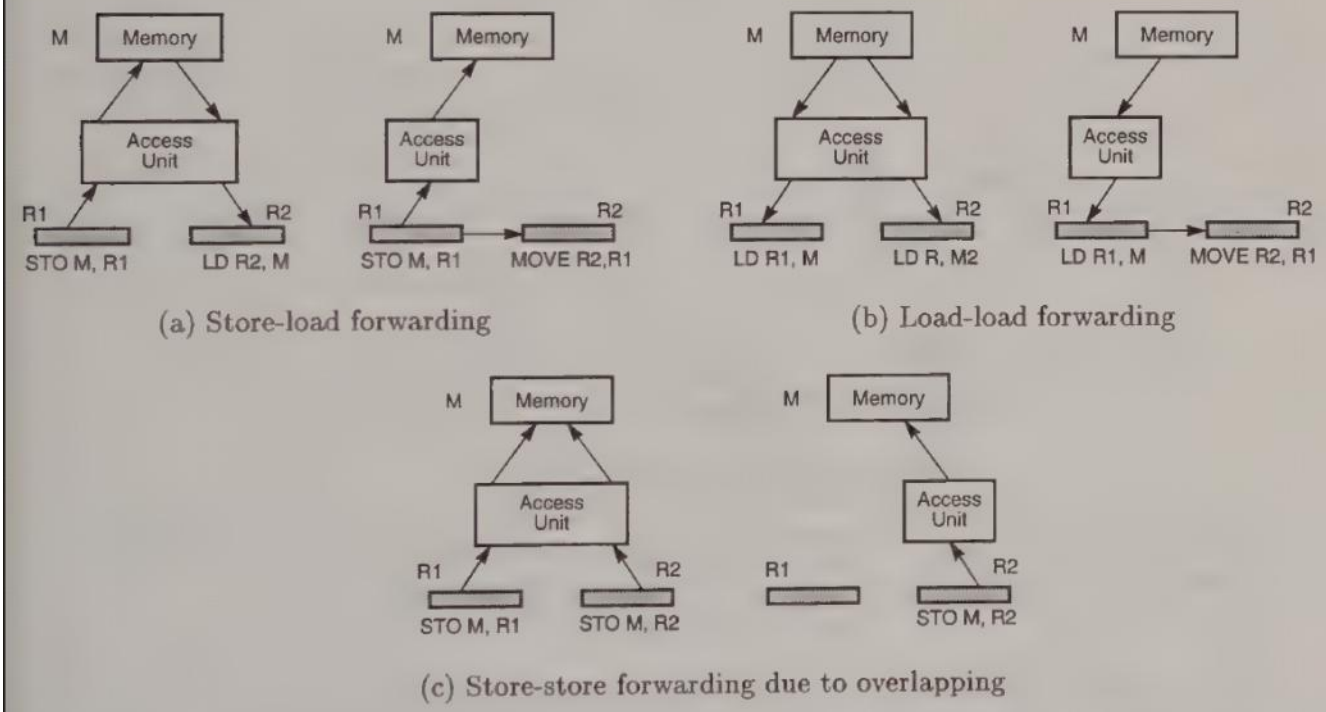


Figure 6.11 The use of sequential and target buffers.



igure 6.12 **A pipelined processor with multiple functional units and distributed reservation stations supported by tagging.** (Courtesy of G. Sohi; reprinted with permission from *IEEE Transactions on Computers*, March 1990)

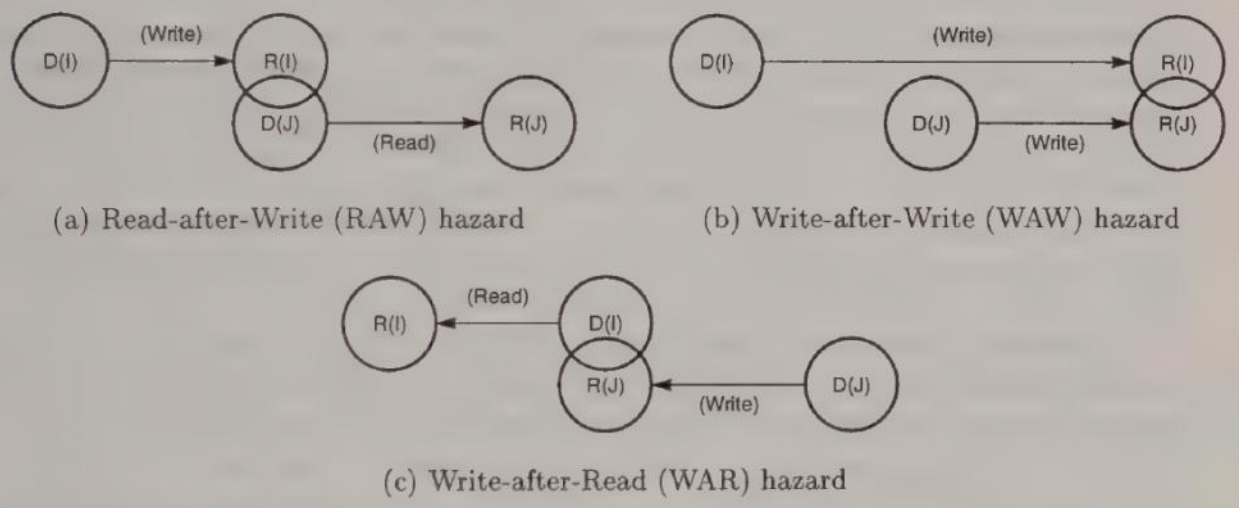


(a) Store-load forwarding

(b) Load-load forwarding

(c) Store-store forwarding due to overlapping

Figure 6.13 Internal data forwarding by replacing memory-access operations with register transfer operations.

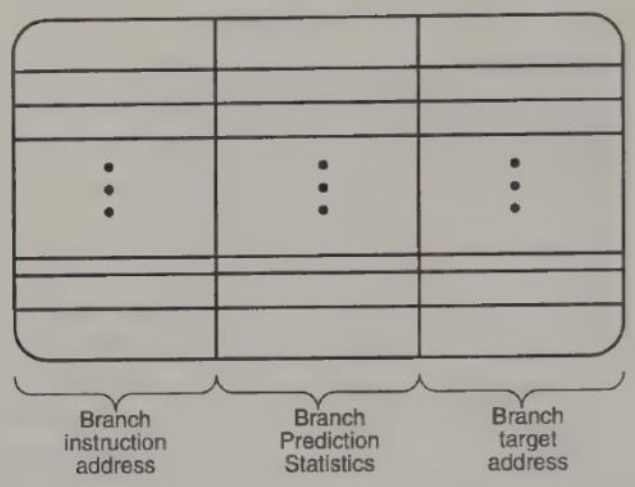


(a) Read-after-Write (RAW) hazard

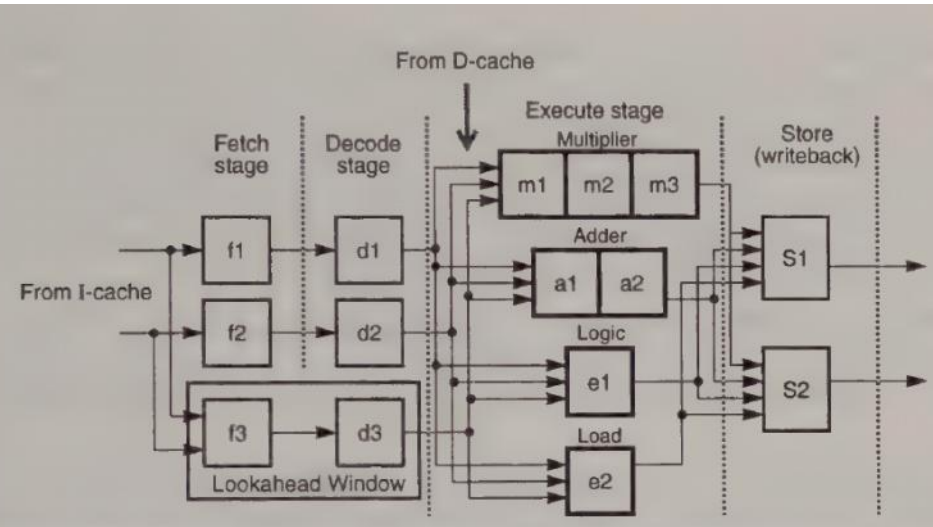
(b) Write-after-Write (WAW) hazard

(c) Write-after-Read (WAR) hazard

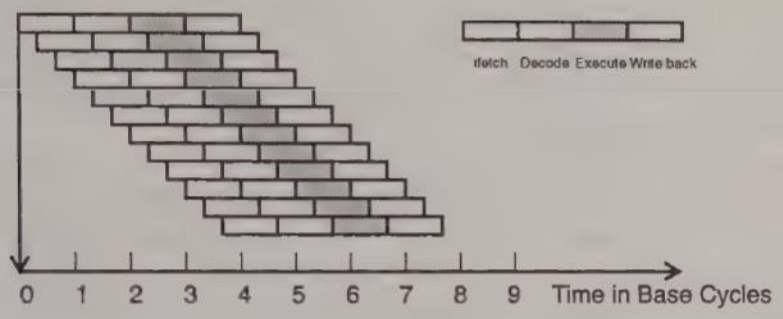
Figure 6.15 Possible hazards between read and write operations in an instruction pipeline (instruction I is ahead of instruction J in program order).



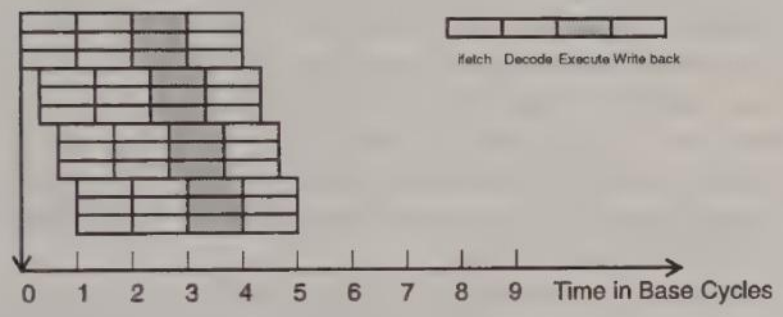
(a) Branch target buffer organization



(a) A dual-pipeline, superscalar processor with four functional units in the execution stage and a lookahead window producing out-of-order issues

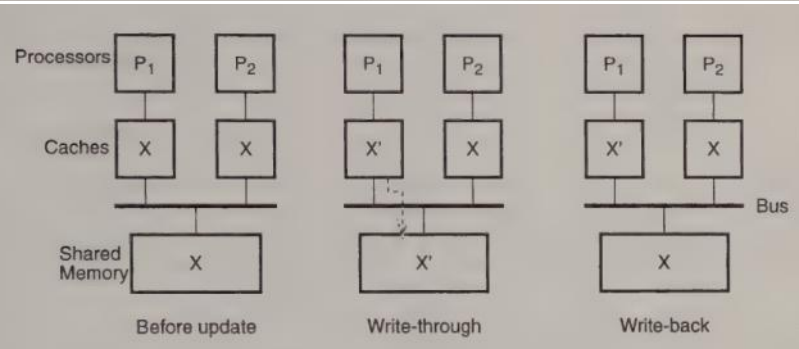


(a) Superpipelined execution with degree $n = 3$

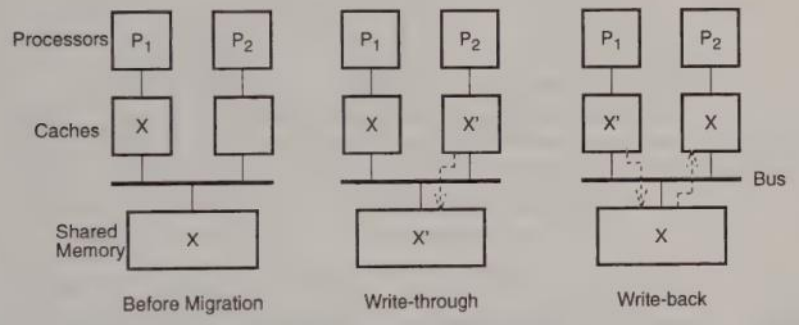


(b) Superpipelined superscalar execution with degree $m = n = 3$

Figure 6.31 Superpipelined processor architectures without and with multiple instruction issues, respectively.

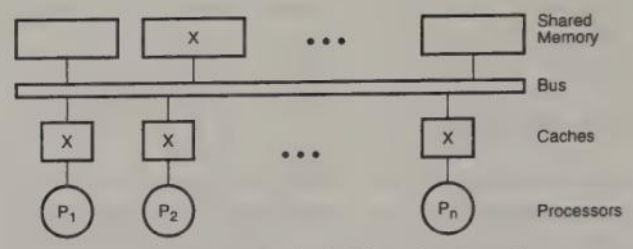


(a) Inconsistency in sharing of writable data

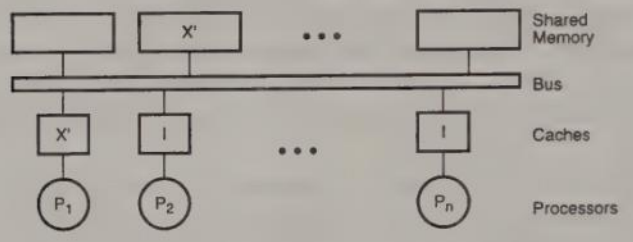


(b) Inconsistency after process migration

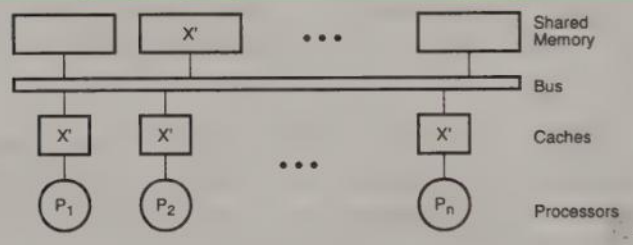
Figure 7.12 Cache coherence problems in data sharing and in process migration. (Adapted from Dubois, Scheurich, and Briggs 1988)



(a) Consistent copies of block X are in shared memory and three processor caches



(b) After a write-invalidate operation by P_1



(c) After a write-update operation by P_1

Figure 7.14 Write-invalidate and write-update coherence protocols for write-through caches (I: invalidate).

Snoopy!
Write once

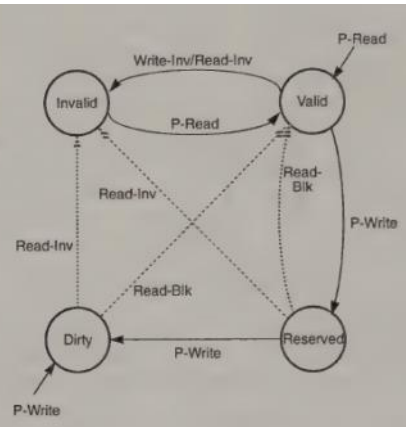


Figure 7.16 Goodman's write-once cache coherence protocol using the write-invalidate policy on write-back caches. (Adapted from James Goodman 1983, reprinted from Stenström, *IEEE Computer*, June 1990)

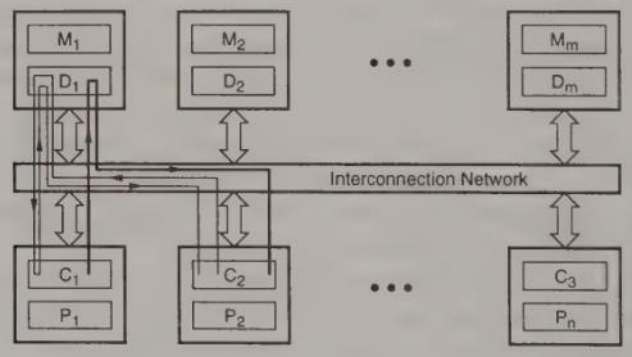
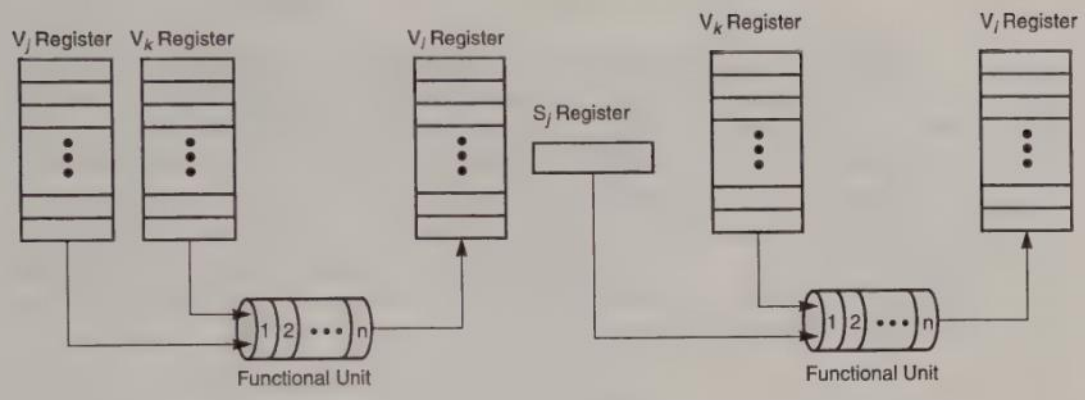
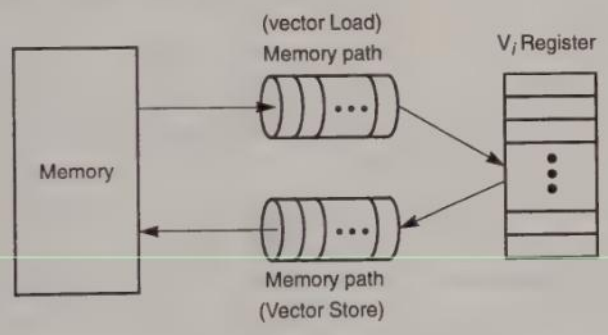


Figure 7.18 Basic concept of a directory-based cache coherence scheme. (Courtesy of Censier and Feautrier, *IEEE Trans. Computers*, Dec. 1978)



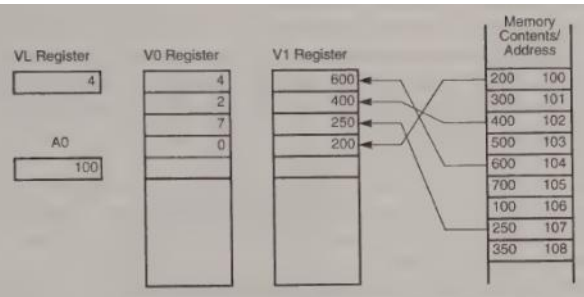
(a) Vector-vector instruction

(b) Vector-scalar instruction

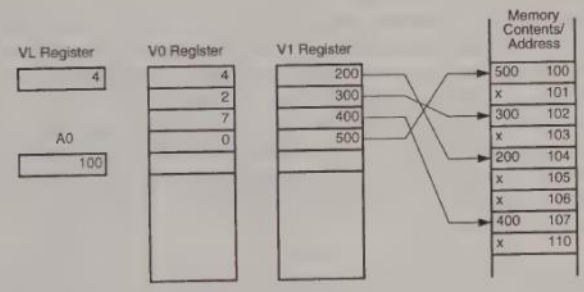


(c) Vector-memory instructions

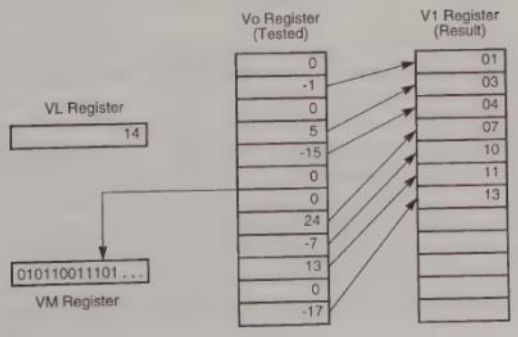
Figure 8.1 Vector instruction types in Cray-like computers.



(a) Gather instruction

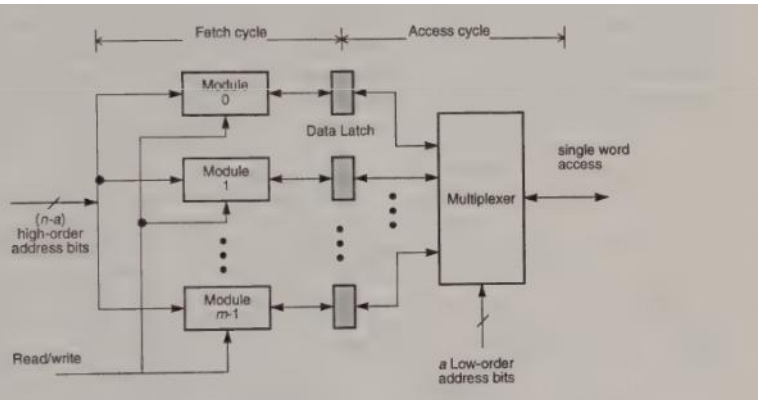


(b) Scatter instruction

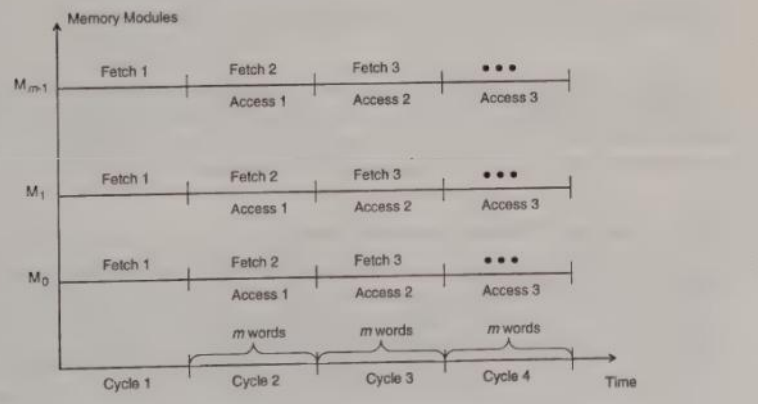


(c) Masking instruction

Figure 8.2 Gather, scatter and masking operations on the Cray Y-MP. (Courtesy of Cray Research Inc., 1990)

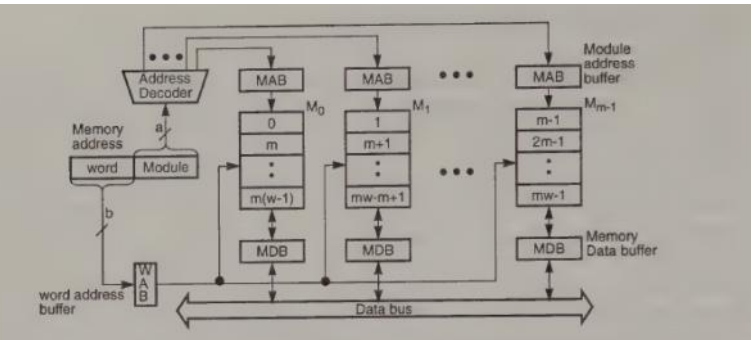


(a) S-access organization for an m-way interleaved memory

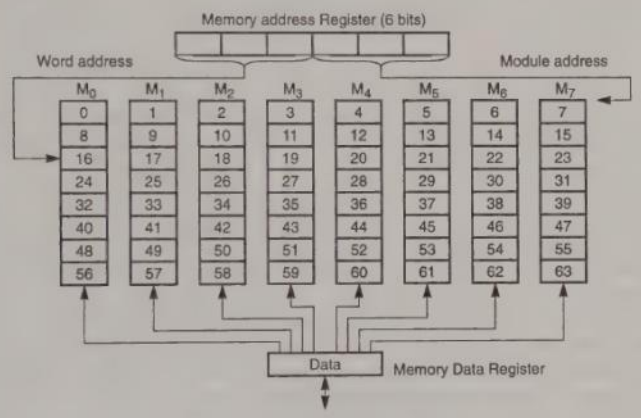


(b) Successive vector accesses using overlapped fetch and access cycles

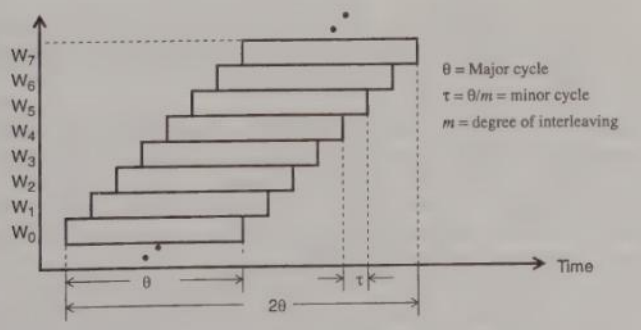
Figure 8.3 The S-access interleaved memory for vector operands access.



(a) Low-order m-way interleaving (the C-access memory scheme)



(a) Eight-way low-order interleaving (absolute address shown in each memory word)



(b) Pipelined access of eight consecutive words in a C-access memory

Figure 5.16 Multiway interleaved memory organization and the C-access timing chart.

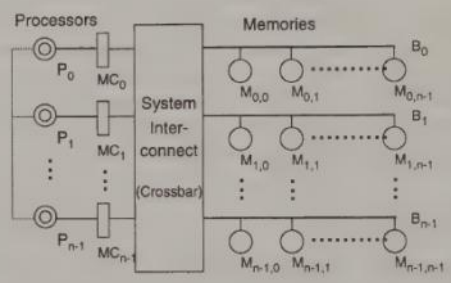


Figure 8.4 The C/S memory organization. (Courtesy of D.K. Panda, 1990)